

WEEK-9

TASK-1: Implement a file copy utility using low-level Linux file 1/0 system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <time.h>

#define BUFFER_SIZE 4096

void low_level_copy(const char *source, const char *destination)
{
    int src, dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;
    src = open(source, O_RDONLY);
    if (src == -1)
    {
        perror("Error opening source file");
        exit(1);
    }
    dest = open(destination, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dest == -1)
    {
        perror("Error opening destination file");
        close(src);
        exit(1);
    }
    if (lseek(src, 0, SEEK_SET) == -1)
    {

```

```

    perror("lseek error");

    close(src);

    close(dest);

    exit(1);
}

while ((bytes_read = read(src, buffer, BUFFER_SIZE)) > 0)
{
    bytes_written = write(dest, buffer, bytes_read);

    if (bytes_written != bytes_read)
    {
        perror("write error");

        close(src);

        close(dest);

        exit(1);
    }
}

if (bytes_read == -1)
    perror("read error");

close(src);

close(dest);
}

void standard_copy(const char *source, const char *destination)
{
    FILE *src, *dest;

    char buffer[BUFFER_SIZE];

    size_t bytes_read;

    src = fopen(source, "rb");

    if (src == NULL)
    {
        perror("Error opening source file");

        exit(1);
    }

```

```

}
dest = fopen(destination, "wb");
if (dest == NULL)
{
    perror("Error opening destination file");
    fclose(src);
    exit(1);
}
while ((bytes_read = fread(buffer, 1, BUFFER_SIZE, src)) > 0)
{
    if (fwrite(buffer, 1, bytes_read, dest) != bytes_read)
    {
        perror("write error");
        fclose(src);
        fclose(dest);
        exit(1);
    }
}
fclose(src);
fclose(dest);
}

int main()
{
    const char *source = "source.txt";
    const char *low_destination = "copy_low.txt";
    const char *std_destination = "copy_std.txt";
    clock_t start, end;
    double low_time, std_time;
    printf("File Copy Performance Comparison\n");
    printf("-----\n");
    start = clock();
    low_level_copy(source, low_destination);
    end = clock();

```

[illegible]

-rw-r--r--	1	root	root	21120	Sep 18 11:51	background_jobs
-rw-r--r--	1	root	root	3288	Sep 18 11:51	background_jobs.c
-rwxr-xr-x	1	root	root	21352	Sep 18 04:07	builtin_shell
-rw-r--r--	1	root	root	3613	Sep 18 04:07	builtin_shell.c
-rw-r--r--	1	root	root	7433	Sep 18 03:52	builtin_shell.c.save
-rwxr-xr-x	1	root	root	21248	Sep 18 04:00	cd_navigation
-rw-r--r--	1	root	root	3683	Sep 18 04:00	cd_navigation.c
-rwxr-xr-x	1	root	root	16464	Sep 16 18:51	client
-rw-r--r--	1	root	root	2123	Sep 16 18:50	client.c
-rw-r--r--	1	root	root	73	Sep 18 09:44	combined.txt
-rwxr-xr-x	1	root	root	29176	Sep 18 09:31	combined_redirect
-rw-r--r--	1	root	root	2939	Sep 18 09:30	combined_redirect.c
-rwxr-xr-x	1	root	root	16576	Aug 17 05:59	command_executor
-rw-r--r--	1	root	root	1252	Aug 17 05:59	command_executor.c
-rwxr-xr-x	1	root	root	26528	Sep 18 04:55	command_history
-rw-r--r--	1	root	root	3589	Sep 18 04:55	command_history.c
-rwxr-xr-x	1	root	root	29472	Sep 18 10:17	complex_command
-rw-r--r--	1	root	root	3948	Sep 18 10:17	complex_command.c
-rwxr-xr-x	1	root	root	16216	Aug 29 10:07	copy
-rw-r--r--	1	root	root	727	Aug 29 09:48	copy.c
-rw-r--r--	1	root	root	727	Aug 29 10:07	copy.c
-rw-r--r--	1	root	root	158	Sep 27 13:25	copy_low.txt
-rw-r--r--	1	root	root	158	Sep 27 13:25	copy_std.txt
-rwxr-xr-x	1	root	root	16184	Aug 29 09:52	copyfile
-rw-r--r--	1	root	root	944	Aug 29 09:51	copyfile.c
-rwxr-xr-x	1	root	root	18688	Sep 17 17:49	cow_demo
-rw-r--r--	1	root	root	1493	Sep 17 17:49	cow_demo.c
-rw-r--r--	1	root	root	29	Sep 18 09:05	date.txt
-rw-r--r--	1	root	root	60	Sep 17 18:18	destination.txt
-rwxr-xr-x	1	root	root	17976	Sep 17 17:41	dynamic_memory
-rw-r--r--	1	root	root	1498	Sep 17 17:41	dynamic_memory.c
-rw-r--r--	1	root	root	60	Sep 18 09:23	error.txt
-rw-r--r--	1	root	root	44	Sep 18 09:23	error2.txt
-rwxr-xr-x	1	root	root	21424	Sep 18 04:14	export_demo
-rw-r--r--	1	root	root	4871	Sep 18 04:13	export_demo.c
-rw-r--r--	1	root	root	0	Aug 29 09:15	file.txt
-rwxr-xr-x	1	root	root	18184	Sep 17 18:14	file_copy
-rw-r--r--	1	root	root	1472	Sep 17 18:13	file_copy.c
-rwxr-xr-x	1	root	root	16568	Sep 27 13:25	filecopy
-rw-r--r--	1	root	root	3335	Sep 27 13:23	filecopy.c
-rwxr-xr-x	1	root	root	16384	Aug 18 03:51	fork_process
-rw-r--r--	1	root	root	1178	Aug 18 03:50	fork_process.c
-rw-r--r--	1	root	root	53	Aug 29 08:13	information.txt
-rw-r--r--	1	root	root	47	Sep 18 18:18	input.txt
-rwxr-xr-x	1	root	root	19488	Sep 18 08:38	input_redirect
-rw-r--r--	1	root	root	1578	Sep 18 08:38	input_redirect.c
-rwxr-xr-x	1	root	root	21912	Sep 18 11:59	job_table
-rw-r--r--	1	root	root	5236	Sep 18 11:59	job_table.c
-rw-r--r--	1	root	root	5025	Sep 18 09:04	listing.txt
-rw-r--r--	1	root	root	1899	Sep 16 06:53	producer_consumer.c
-rwxr-xr-x	1	root	root	20288	Sep 18 04:11	pwd_exit
-rw-r--r--	1	root	root	3058	Sep 18 04:11	pwd_exit.c
-rw-r--r--	1	root	root	47	Sep 18 11:46	result.txt
-rw-r--r--	1	root	root	52	Sep 17 18:22	sample.txt
-rwxr-xr-x	1	root	root	16664	Sep 16 18:50	server
-rw-r--r--	1	root	root	3884	Sep 16 18:49	server.c
-rwxr-xr-x	1	root	root	16488	Sep 16 19:44	signal_handler
-rw-r--r--	1	root	root	2823	Sep 16 19:43	signal_handler.c
-rw-r--r--	1	root	root	158	Sep 27 13:25	source.txt
-rwxr-xr-x	1	root	root	20128	Sep 18 09:22	stderr_redirect
-rw-r--r--	1	root	root	2788	Sep 18 09:22	stderr_redirect.c
-rwxr-xr-x	1	root	root	16312	Aug 12 06:45	task1
-rw-r--r--	1	root	root	555	Aug 12 06:44	task1.c
-rwxr-xr-x	1	root	root	16224	Aug 12 06:55	task2
-rw-r--r--	1	root	root	1848	Aug 12 06:55	task2.c
-rwxr-xr-x	1	root	root	16232	Aug 12 07:01	task3
-rw-r--r--	1	root	root	1715	Aug 12 07:01	task3.c
-rwxr-xr-x	1	root	root	16696	Aug 12 07:10	task4
-rw-r--r--	1	root	root	4219	Aug 12 07:10	task4.c
-rwxr-xr-x	2	root	root	4896	Aug 17 05:19	task5
-rw-r--r--	1	root	root	3884	Aug 12 08:16	task5.c
-rwxr-xr-x	1	root	root	18912	Aug 12 08:11	task5_old
-rwxr-xr-x	1	root	root	16488	Aug 3 05:53	task6
-rw-r--r--	1	root	root	48	Aug 20 08:19	text.txt
-rw-r--r--	1	root	root	8131	Sep 17 18:23	trace.txt
-rw-r--r--	1	root	root	5	Sep 18 09:06	user.txt
-rwxr-xr-x	1	root	root	19376	Sep 18 03:40	variable_expand
-rw-r--r--	1	root	root	4688	Sep 18 03:40	variable_expand.c
-rwxr-xr-x	1	root	root	18152	Sep 17 19:02	waitpid_demo
-rw-r--r--	1	root	root	1685	Sep 17 19:02	waitpid_demo.c
-rwxr-xr-x	2	root	root	4896	Aug 8 08:14	week3
root@LAPTOP-RJFDBE8T:~#	cat	source.txt				
This is a test file for Linux file copy system calls.						
root@LAPTOP-RJFDBE8T:~#	cat	copy_low.txt				
This file is used to compare low level and standard library I/O. This is my OSSP system call experiment.						
root@LAPTOP-RJFDBE8T:~#	cat	copy_std.txt				
This file is used to compare low level and standard library I/O. This is my OSSP system call experiment.						
root@LAPTOP-RJFDBE8T:~#	cat	copy_low.txt				
root@LAPTOP-RJFDBE8T:~#	cat	copy_std.txt				
root@LAPTOP-RJFDBE8T:~#	wc	-c	source.txt	copy_low.txt	copy_std.txt	
158	source.txt					
158	copy_low.txt					
158	copy_std.txt					
474	total					
root@LAPTOP-RJFDBE8T:~#	ls					

TASK-2: Develop a program that redirects standard input and Output using file descriptors and dup2(). Demonstrate how shells implement 1/0 redirection.

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <fcntl.h>

#include <unistd.h>

int main()
{
    int fd;

    fd = open("input.txt", O_RDONLY);

    if (fd < 0)
    {
        perror("open input.txt");

        exit(1);
    }

    if (dup2(fd, STDIN_FILENO) < 0)
    {
        perror("dup2 stdin");

        exit(1);
    }

    close(fd);
```

```

fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd < 0)
{
    perror("open output.txt");
    exit(1);
}
if (dup2(fd, STDOUT_FILENO) < 0)
{
    perror("dup2 stdout");
    exit(1);
}
close(fd);
char buffer[100];
ssize_t n = read(STDIN_FILENO, buffer, sizeof(buffer) - 1);
if (n < 0)
{
    perror("read");
    exit(1);
}
buffer[n] = '\0';
printf("Data read from input.txt:\n");
printf("%s\n", buffer);
return 0;
}

```

```

root@LAPTOP-RJFD8EBT:~# nano redirect.c
root@LAPTOP-RJFD8EBT:~# nano input.txt
root@LAPTOP-RJFD8EBT:~# gcc redirect.c -o redirect
root@LAPTOP-RJFD8EBT:~# ./redirect
root@LAPTOP-RJFD8EBT:~# cat output.txt
Data read from input.txt:
Hello, this is input redirection.
This text is being read from a file.

root@LAPTOP-RJFD8EBT:~# cat < input.txt
Hello, this is input redirection.
This text is being read from a file.
root@LAPTOP-RJFD8EBT:~# cat input.txt > test.txt
root@LAPTOP-RJFD8EBT:~# cat test.txt
Hello, this is input redirection.
This text is being read from a file.
root@LAPTOP-RJFD8EBT:~# cat < input.txt > test2.txt
root@LAPTOP-RJFD8EBT:~# cat test2.txt
Hello, this is input redirection.
This text is being read from a file.
root@LAPTOP-RJFD8EBT:~# |

```

